OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

Studie zur verbesserten Anomalyerkennung mit erweiterter Kontextinformation

Research work
of

Yannic Hanel
3296954

curriculum
at the

Ostbayerischen Technischen Hochschule Regensburg
Faculty of Computer Science and Mathematics

Advisor: Prof. Dr. Sebastian Fischer

Regensburg, 20.02.2026

Contents

Abkürzungsverzeichnis **4**

1 Einleitung **1**
1.1 Motivation und Problemstellung 1
1.2 Zielsetzung der Arbeit 1
1.3 Aufbau der Arbeit 1

2 Related Work **3**

3 Systemarchitektur und Detection Framework **4**
3.1 Systemüberblick und verteilte Kommunikation 4
3.1.1 Privates IPFS-Netzwerk und Peer-to-Peer-Datenaustausch 5
3.1.2 End-to-End-Datenfluss 5
3.2 Physische Testumgebung 7
3.3 Detection Einheiten der Ladeinfrastruktur 8
3.3.1 System- und Leistungsanomaliedetektion (pi_anomaly_detection) 9
3.3.2 Physikalische Signal- und Temperaturüberwachung (detection_power_temperature) 9
3.3.3 KI-basierte Angriffserkennung im Netzwerk (ids_power_ai_detection) . . 10
3.3.4 Hardware-Ereignisüberwachung (detection_hardwareevents) 10

4 Konzeption des Threat Reporters **11**
4.1 Zentralisiertes Datenmanagement und Merkmalsextraktion für den Threat Report 11
4.2 Zeitliche Diskretisierung und Intervalldefinition 11
4.3 Statistische Merkmalsaggregation mittels Sliding-Window-Analyse 12

5 Theoretische Grundlagen der Informationsfusion **13**
5.1 Vergleich von Aggregationsmethoden 13
5.1.1 Dempster-Shafer-Evidenztheorie 13
5.1.2 Fuzzy-basierte Fusion 13
5.1.3 Bayes-basierte Konsensmodelle 13
5.1.4 Gewichtetes Voting 13
5.2 State-of-the-Art: Deep Sets und Attention Mechanismen 13

6 Methodik: Gewichtete Aggregation mittels Utility Theory **14**
6.1 Anforderungen an die Gewichtungsfunktion 14
6.2 Mathematische Modellierung 14
6.2.1 Die Gewichtungsformel: $w(n) = BASE_WEIGHT + (MAX_WEIGHT - BASE_WEIGHT) \times (1 - \exp(-K \cdot n))$ 14
6.2.2 Parameteranalyse: Monotonie und Diminishing Returns 14
6.2.3 Abgrenzung zur originären Utility Theory 14
6.3 Implementierung des Attention-basierten Selektionsmodells 14

7 Erweiterung der Systemarchitektur und Implementierung **15**
7.1 Generierte Threat Reports: Format und Struktur 15

7.2 Integration von Threat Reporter und Aggregationslogik 15
7.3 Architektur der Fusionskomponente im P2P-Netzwerk 15

8 Evaluation **16**
8.1 Qualität der aggregierten Threat Reports 16
8.2 Nützlichkeit für die interne Nutzung der Knoten 16

9 Diskussion und Forschungslücken **17**
9.1 Auswirkungen unterschiedlicher Gewichtungsstrategien auf die Report-Qualität . 17
9.2 Potenzial der LLM-gestützten Aufbereitung für externe CTI-Nutzung 17
9.3 Untersuchung des Einflusses von Report-Aggregation auf Detectionsergebnisse (Echo-Chamber-Effekt?) 17

10 Zusammenfassung und Fazit **18**

11 Quellcode **19**

Bibliography **22**

Abkürzungsverzeichnis

DNN	Deep Neural Network
DoS	Denial of Service
IPFS	InterPlanetary File System
P2P	Peer-to-Peer
CID	Content Identifier
MADDR	Multiaddress
CTI	Cyber Threat Intelligence
I2C	Inter-Integrated Circuit
MSE	Mean Squared Error
LSTM	Long Short Term Memory

1 Einleitung

1.1 Motivation und Problemstellung

Obwohl die node-basierten Detektionsmechanismen der betrachteten Ladeinfrastruktur bereits eine beachtliche Erkennungsleistung aufweisen, besteht das Ziel, die Spezifität (True-Negative-Rate) sowie die Robustheit des Gesamtsystems weiter zu steigern. Lokale Anomalieerkennungsverfahren stoßen systembedingt an Grenzen, wenn netzwerkweite Veränderungen oder subtile, koordinierte Manipulationsversuche vorliegen, die auf Ebene einer einzelnen Wallbox nicht als bösartig identifiziert werden können.

Quelle evtl. mit meiner Bachelorarbeit arbeiten

Quelle schwierige erkennung wenn netzwerkweite Veränderung und nicht am Knoten selbst

Die Motivation dieser Arbeit liegt daher in der Entwicklung einer Methodik, die es einzelnen Ladepunkten ermöglicht, Kontextinformationen benachbarter Knoten in den Entscheidungsprozess einzubeziehen. Es wird vermutet, dass durch diesen kollaborativen Ansatz räumliche und zeitliche Korrelationen innerhalb des Netzwerks besser erfasst werden können, die über die isolierte Betrachtung eines einzelnen Knotens hinausgehen. Durch die Integration dieser Kontextdaten wird erhoft eine präzisere Differenzierung zwischen globalen Schwankungen und lokalen Sicherheitsvorfällen zu ermöglichen. Es wird erwartet, dass dieser erweiterte Kontextbezug zu einer Reduktion von Fehlalarmen (False Positives) sowie einer verbesserten Erkennungsrate tatsächlicher Bedrohungen (True Positives) beiträgt. Inwiefern diese Annahmen empirisch belastbar sind, bleibt jedoch offen und soll in einer weiterführenden Arbeit durch die Evaluation anhand eines unabhängigen Datensatzes systematisch untersucht werden.

1.2 Zielsetzung der Arbeit

Um die dargelegte Problemstellung zu adressieren, liegt der Fokus dieser Arbeit auf der Konzeption und Implementierung einer robusten Infrastruktur für die Anomalieerkennung innerhalb der Ladeinfrastruktur. Übergeordnetes Ziel ist die Schaffung einer stabilen Betriebsumgebung, die einen zuverlässigen, kontextbasierten Datenaustausch zwischen den einzelnen Knoten gewährleistet.

Ein wesentlicher Aspekt hierbei ist die Realisierung einer konsistenten und effizienten Aggregation dezentraler Informationen, um die Detektionsgüte der beteiligten Entitäten signifikant zu steigern. Nur durch eine performante Umgebung, welche den reibungslosen Austausch von Kontextinformationen sicherstellt, kann die in Kapitel 3 (Systemarchitektur und Detection Framework) detaillierte Anomalieerkennung erfolgreich implementiert und langfristig wartbar betrieben werden. Die Entwicklung dieser stabilen Kommunikations- und Verarbeitungsplattform bildet somit den zentralen Kernbestandteil der vorliegenden Arbeit.

1.3 Aufbau der Arbeit

Zunächst wird die Systemarchitektur im zur Verfügung stehenden Peer-to-Peer-Netzwerk beschrieben sowie die implementierten Detektionen und die Datenakquise über Prometheus empirisch erläutert und differenziert.

Darauf folgend wird die Konzeption des Threat Reporters vorgestellt, welcher die Detektionsergebnisse in einem standardisierten Format aggregiert und über definierte Schnittstellen zwischen den Knoten überträgt.

Anschließend werden die theoretischen Grundlagen der Informationsfusion präsentiert.

Darauf folgend wird die Methodik der gewichteten Aggregation mittels Utility Theory beschrieben.

Anschließend wird die erweiterte Architektur und umsetzung der Vorgestellten Methodik beschrieben. Die Daraus generierten Threat Reports werden präsentiert und hinsichtlich ihrer Qualität und Nützlichkeit für die interne Nutzung der Knoten bewertet.

Die anschließende Diskussion der Ergebnisse führt zur Identifikation einer Forschungslücke, welche in zukünftigen Arbeiten weiter untersucht werden sollte.

Vielleicht

TODO: Sollte hier wissenschaftlich begründet werden, warum die Wahl auf eine Kombination aus Deep Sets und Attention-Mechanismen gefallen ist?

TODO: Die Formel der Utility Theory wurde nicht originär für Gewichtungsfunktionen entwickelt. Wie problematisch ist das? Sollte dies in der Diskussion thematisiert oder direkt eine alternative Methode verwendet werden?

TODO: Fokus darauf legen wie die zuvor beschriebenen Methoden in die Architektur einbepflegt wurden

Welche Forschungslück

TODO: Mögliche Forschungslück

2 Related Work

3 Systemarchitektur und Detection Framework

3.1 Systemüberblick und verteilte Kommunikation

Das ReSiLENT-System gliedert sich in zwei logische Domänen: die **Charging Station** mit dem eingebetteten Detection-Client sowie das **Cloud Dashboard** zur zentralen Visualisierung und Auswertung. Beide Domänen sind über ein privates Peer-to-Peer-Netzwerk auf Basis des *Inter-Planetary File System* (IPFS) lose gekoppelt. Anstelle direkter IP-zu-IP-Verbindungen erfolgt der Datenaustausch über inhaltsadressierte Bezeichner (*Content Identifier*, CIDs).

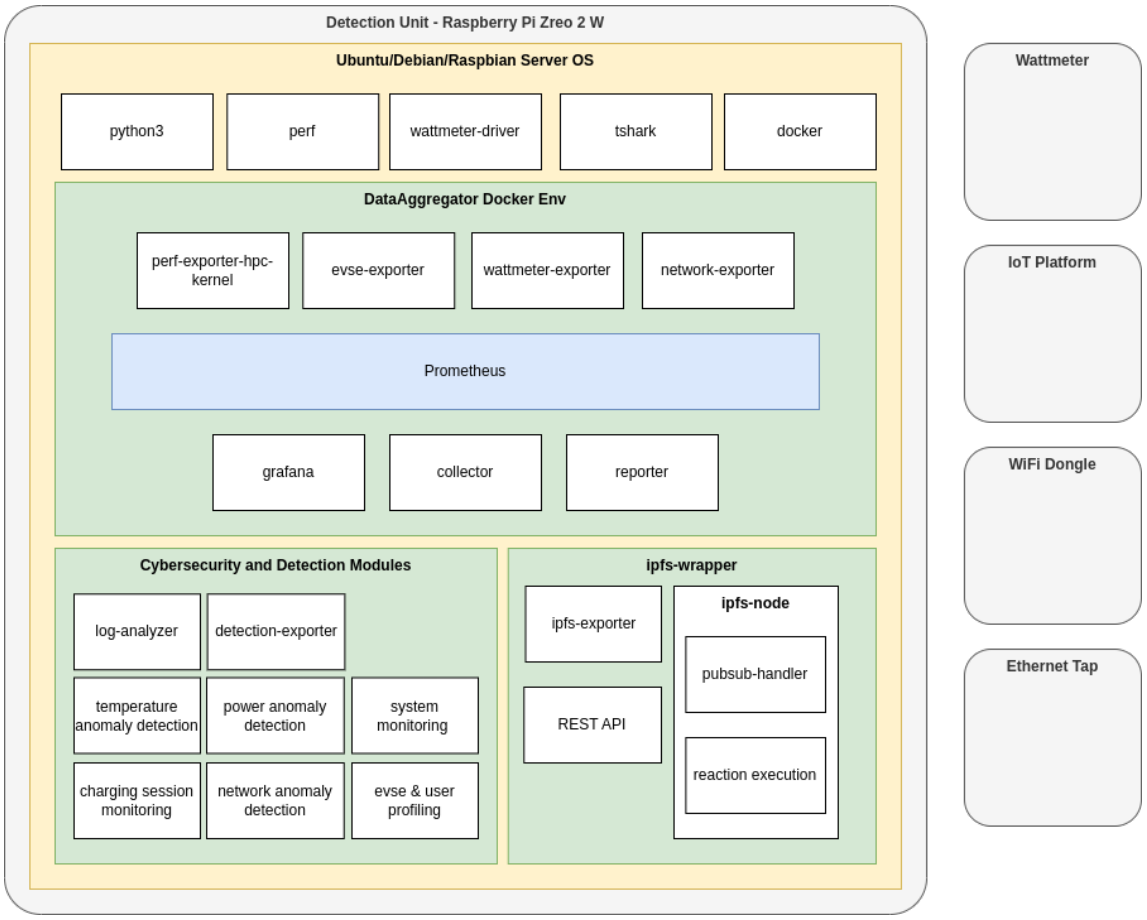

Figure 1: Gesamtarchitektur des ReSiLENT-Systems mit Charging Station und Cloud Dashboard

Abbildung 1 zeigt die Gesamtarchitektur des Systems mit Fokus auf die in jeder Wallbox eingebettete Detection Unit. Als zentrales Laufzeitmodul fungiert der *DataAggregator*, der die erfassten Sensordaten der Wallbox über Prometheus bereitstellt und sowohl den Cybersecurity- und Detection-Modulen als auch Werkzeugen zur Datenaufbereitung zugänglich macht.

Die Architektur ist auf kollaborative Angriffserkennung ausgelegt: Über den *ipfs-wrapper* und dessen **Publish-Subscribe-Mechanismus** (detailliert in Abbildung 2) teilen Ladesäulen Kontextinformationen mit allen Netzwerkteilnehmern und können somit die situative Wahrnehmung über die gesamte Ladeinfrastruktur hinweg verbessern.

3.1.1 Privates IPFS-Netzwerk und Peer-to-Peer-Datenaustausch

Für die verteilte Kommunikation zwischen den Detection Units und dem Cloud Dashboard wird ein **privates IPFS-Netzwerk** eingesetzt. Im Gegensatz zum öffentlichen IPFS-Netz wird der Zugang durch einen gemeinsamen *Swarm Key* beschränkt, sodass ausschließlich autorisierte Knoten dem Netzwerk beitreten können. Die Netzwerktopologie ist in Abbildung 2 dargestellt.

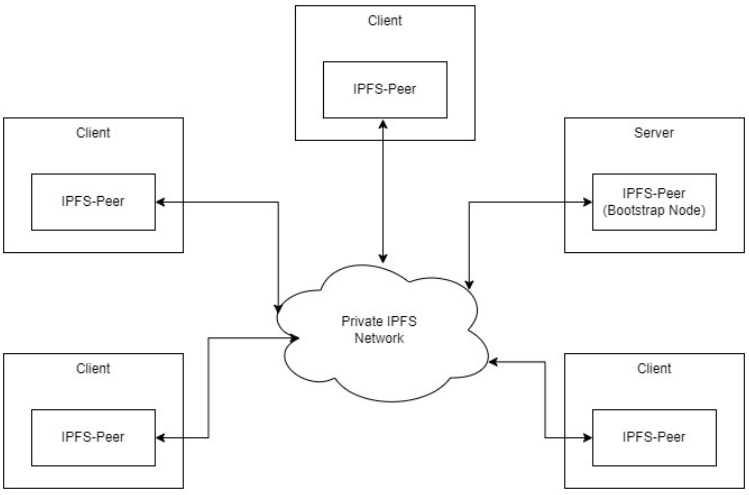

Figure 2: Topologie des privaten IPFS-Netzwerks mit Bootstrap-Node und Client-Peers

Der Netzbeitritt erfolgt über einen dedizierten **Bootstrap-Node**, der auf dem Server des Cloud Dashboards betrieben wird. Neue Peers melden sich beim Bootstrap-Node an, erhalten die Multiaddresses (*MADDR*) der bekannten Peers und kommunizieren anschließend direkt miteinander, ohne den Bootstrap-Node als Relay zu verwenden. Jeder Knoten betreibt einen lokalen **IPFS-Peer-Daemon**, der über den `ipfs-wrapper`, wie in Abbildung 3 dargestellt, in die Systemarchitektur eingebunden ist.

Für die Benachrichtigung über neue Daten wird der **Publish-Subscribe-Mechanismus** von IPFS genutzt: Sobald neue Messdaten erhoben werden, können diese veröffentlicht werden indem sie die zugehörige CID zusammen mit der eigenen MADDR über einen gemeinsam abonnierten Kanal an alle Teilnehmer versendet. Die Empfänger rufen die Datei daraufhin direkt beim Ursprungs-Peer ab. Da eine CID dem kryptografischen Hash des jeweiligen Inhalts entspricht, ist die Datenintegrität implizit gewährleistet, ohne dass zusätzliche Signaturmechanismen erforderlich sind.

3.1.2 End-to-End-Datenfluss

Der vollständige Datenfluss vom Sensor bis zur Visualisierung lässt sich in drei Phasen unterteilen: Datenerhebung und Aggregation auf der Charging Station, Transport über das IPFS-Netz sowie Persistierung und Darstellung im Cloud Dashboard. Das Sequenzdiagramm in Abbildung 4 sowie der Datenfluss in Abbildung 3 illustrieren diesen Ablauf.

Datenerhebung (Charging Station)

Der *Aggregator* innerhalb des Knotens sammelt die Messdaten. Prometheus scraped alle Exporter in konfigurierbaren Intervallen. Der *Reporter* liest die aggregierten Daten aus der Prometheus-

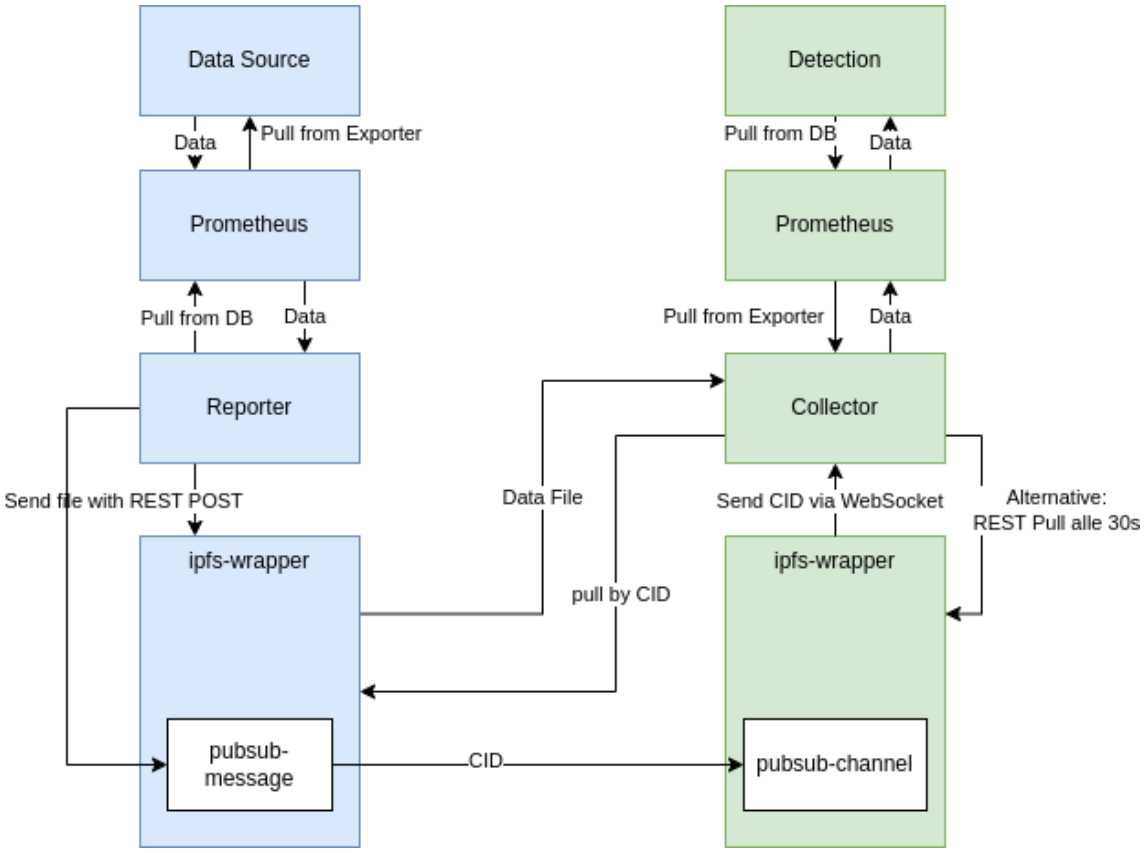

Figure 3: Datenfluss zwischen Charging Station (blau) und Cloud Dashboard (grün) über IPFS

Datenbank und übermittelt sie an den `ipfs-wrapper`.

Transport (IPFS-Netzwerk)

Der `ipfs-wrapper` nimmt die Datei entgegen, publiziert sie im lokalen IPFS-Node und erhält eine CID zurück. Anschließend wird die CID zusammen mit der eigenen MADDR über den konfigurierten **Publish-Subscribe-Mechanismus** im privaten IPFS-Netz verbreitet. Die eigentlichen Messdaten werden dabei nicht aktiv gepusht, lediglich der Zeiger (CID) wird distribuiert.

Persistierung und Visualisierung (Cloud Dashboard)

Auf der Serverseite empfängt der *Publish-Subscribe-Event-Listener* des `ipfs-wrapper` die CID und MADDR und leitet diese per **WebSocket** an den Collector weiter. Der Collector fordert daraufhin die vollständige Datei über den lokalen IPFS-Peer an, der sie in Chunks vom Ursprungs-Peer abruft. Der *File Extractor* verarbeitet die Rohdaten und persistiert die Messwerte in **InfluxDB**. Grafana greift auf InfluxDB zu und stellt die Zeitreihendaten im Dashboard dar (vgl. Abbildung 5).

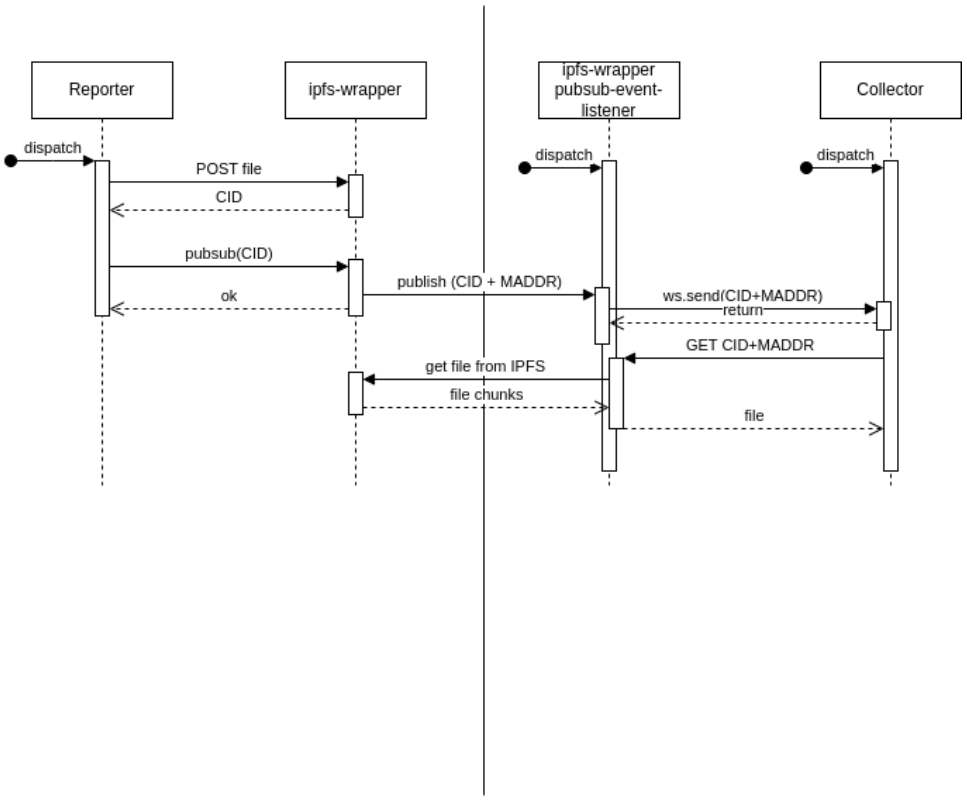

Figure 4: Sequenzdiagramm des IPFS-basierten Datenaustausches

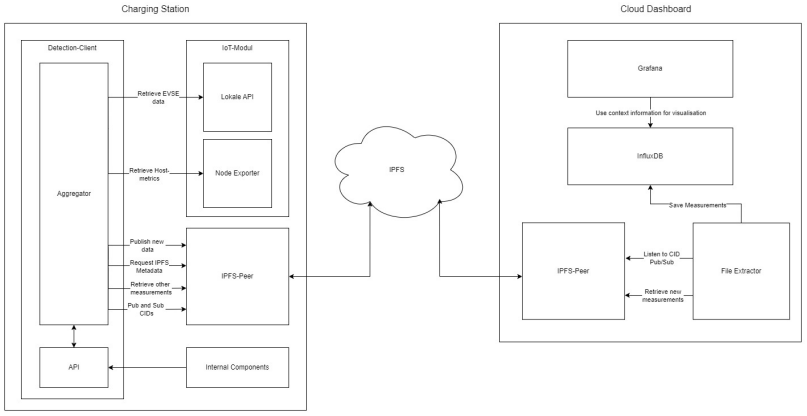

Figure 5: Komponenten des Cloud Dashboards: IPFS-Peer, File Extractor, InfluxDB und Grafana

Dieses Verständnis der Systemarchitektur und des Datenflusses bildet die Grundlage für die folgende Arbeit zur Implementierung des CTI-Reports und später des aggregierten CTI-Reports.

3.2 Physische Testumgebung

Die physische Testumgebung für die Implementierung und Evaluierung der vorliegenden Arbeit besteht aus einer realen ChargeIQ-Wallbox. Die Detection-Module sind in diesem Szenario abweichend von der in Kapitel "Systemüberblick und verteilte Kommunikation" 3.1 beschriebenen

Zielarchitektur nicht auf einem Raspberry Pi Zero 2 W, sondern zu Testzwecken auf einem leistungsfähigeren Raspberry Pi 4 B oder BananaPi M5 installiert. Diese sind über eine kabelgebundene Ethernet-Verbindung in das private IPFS-Netzwerk eingebunden, um eine stabile und performante Kommunikation zu gewährleisten. Die Wallbox selbst ist ebenfalls über eine drahtlose Verbindung in dasselbe private IPFS-Netzwerk integriert.

Darüber hinaus besteht eine serielle *I2C*-Verbindung (*Inter-Integrated Circuit*) zwischen der ChargeIQ-Wallbox und einem Einplatinencomputer in diesem fall der Raspberry Pi 4 B, welcher stellvertretend für die spätere Detection Unit auf Basis des Raspberry Pi Zero 2 W fungiert. Diese Verbindung ermöglicht die Erfassung von Echtzeitdaten des integrierten Wattmeters, welcher die sensor daten für eine im Kapitel "Detection Units der Ladeinfrastruktur" **??** beschriebene Detektionseinheit bereitstellt.

Zusätzliche Sensordaten werden dem Raspberry Pi 4 B über die in der Testumgebung eingerichtete drahtlose WLAN-Verbindung übermittelt, in welche sowohl die als Detection Unit eingesetzte Einplatinencomputer als auch die Wallboxen eingebunden sind.

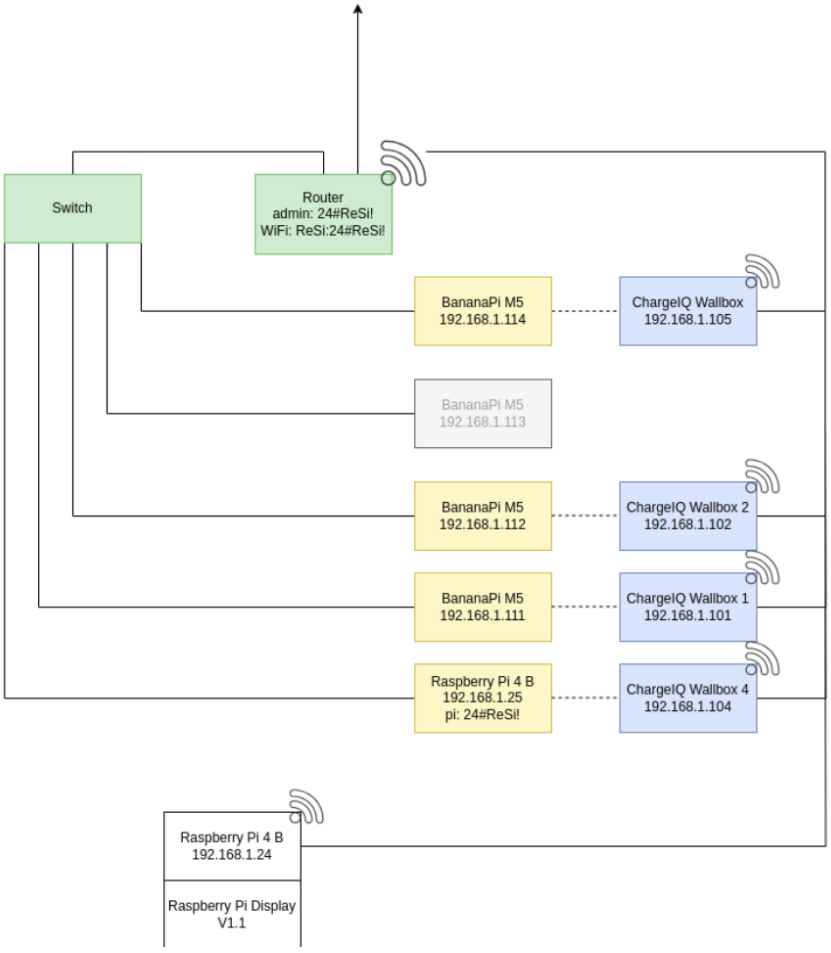

Figure 6: Schematische Darstellung der physischen Testumgebung.

3.3 Detection Einheiten der Ladeinfrastruktur

Im Folgenden werden die vier Hauptdetektionseinheiten der Testumgebung der Ladeinfrastruktur vorgestellt, die im Rahmen dieses Projekts implementiert wurden und die für den Cyber Threat

Report sowie den Aggregierten Cyber Threat Report von zentraler Bedeutung sind.

3.3.1 System- und Leistungsanomaliedetektion (pi_anomaly_detection)

Die *Pi Anomaly Detection Einheit* trifft Entscheidungen anhand der folgenden Metriken: Shunt-Spannung, Bus-Spannung, Stromstärke und Leistung. Diese Messdaten werden von den Sensoren der Wallbox erfasst und an Prometheus weitergeleitet. Die beschriebene Anomaliedetektionseinheit greift diese Daten über Prometheus auf und nutzt daraufhin zwei Deep-Learning-Modelle:

- **xLSTM-Autoencoder:** Dieses Modell operiert auf einem Sliding-Window von $w = 44$ Sekunden. Es lernt die Normalverteilung der physikalischen Parameter (Spannung, Strom, Leistung). Eine Anomalie wird detektiert, wenn der Rekonstruktionsfehler E_t den Schwellenwert $\tau = 0.008$ überschreitet. Die Wahrscheinlichkeit wird über eine Sigmoid-Funktion approximiert:
$$P_{xlstm} = \frac{1}{1 + e^{-50 \cdot (E_t - \tau)}}$$
- **Transformer-Klassifikator:** Parallel dazu extrahiert das System mittels *tsfresh* 15 statistische Merkmale aus einem 60-Sekunden-Fenster. Ein Transformer-Modell klassifiziert diese aggregierten Merkmale direkt in die Kategorien *benign* oder *attack*.
- **Sequentielles Voting-Verfahren:** Zur Stabilisierung der Detektion nutzt das System einen *Deques*-Puffer der Länge $k = 3$. Ein Alarm wird nur ausgelöst, wenn:
  - Beide Modelle im aktuellsten Zeitschritt eine Anomalie melden ODER
  - Mindestens eines der Modelle eine konsistente Sequenz von drei aufeinanderfolgenden positiven Detektionen aufweist.

Die Detektionseinheit reagiert insbesondere auf Stromspitzen und -einbrüche, welche primär über die genannten Metriken identifiziert werden können.

3.3.2 Physikalische Signal- und Temperaturüberwachung (detection_power_temperature)

Diese Detektionseinheit arbeitet mit einer klassischen regelbasierten Logik, welche IDS-Muster zuordnet und auf Basis von Schwellenwerten für physikalische Parameter (Spannung, Leistung, Temperatur) Anomalien detektiert. Die Detektion ist in zwei Bereiche der IDS-Mustererkennung unterteilt:

- **Temperaturüberwachung:** Diese Anomalieerkennung ruft die Temperaturdaten der letzten 60 Sekunden von Prometheus ab und erkennt Anomalien anhand von drei definierten Regeln:
  - *Absolute Obergrenze:* Wenn die Temperatur im abgefragten Zeitfenster einen Wert von 70 °C überschreitet, wird eine Anomalie detektiert.
  - *Konstantreihe + Sprung:* Wenn ein Temperatursprung bei einer ansonsten konstanten Reihe eine Toleranz von ± 1 überschreitet, wird eine Anomalie gemeldet.
  - *Anomalie durch Z-Score:* Wenn die Standardabweichung der Temperaturwerte einen Z-Score von $> 3,0$ erreicht, löst die Anomalieerkennung aus.

- **Wattmeter-Überwachung:** Analog zur System- und Leistungsanomaliedetektion (siehe Abschnitt 3.3.1) ruft diese Detection ebenfalls vier Metriken zu Spannung und Leistung der letzten 60 Sekunden ab. Die Detektion prüft, ob diese Werte fest definierte Schwellenwerte (Minimum oder Maximum) unter- bzw. überschreiten. Ist dies der Fall, wird eine Anomalie ausgelöst.

Sobald die Temperaturüberwachung und/oder die Wattmeterüberwachung eine positive Anomalie detektieren, schlägt auch der kombinierte Alarm *detection_power_temperature* aus.

3.3.3 KI-basierte Angriffserkennung im Netzwerk (ids_power_ai_detection)

Diese Detection-Komponente analysiert dieselben vier physikalischen Messgrößen, die in Abschnitt 3.3.1 beschrieben wurden, verzichtet jedoch vollständig auf statische Schwellenwerte. Stattdessen kommen zwei komplementäre Machine-Learning-Modelle zum Einsatz, die anomales Verhalten im zeitlichen Kontext erkennen:

- *Isolation Forest:* Dies ist ein Verfahren, welches einzelne Messpunkte anhand ihrer statistischen Isolation im Merkmalsraum bewertet. Für jeden Messzeitpunkt wird ein Anomalie-Score berechnet. Weicht ein Datenpunkt signifikant vom gelernten Normalzustand ab, wird dies als Anomalie gewertet.
- *LSTM-Autoencoder:* Wie für LSTM-Modelle (Long Short-Term Memory) üblich, betrachtet dieses Modell nicht nur einzelne Messwerte, sondern eine Sequenz von 60 Sekunden (bei einem Abfrageintervall von 5 Sekunden). Das Modell wurde auf normalem Betriebsverhalten trainiert und lernt, typische Zeitreihenmuster zu rekonstruieren. Zur Erkennung wird der Mean Squared Error (MSE) zwischen der originalen Sequenz und ihrer Rekonstruktion berechnet. Überschreitet der MSE des aktuellen Fensters einen empirisch bestimmten Schwellenwert (`LSTM_THRESHOLD` = 0,01), wird eine Anomalie ausgelöst.

3.3.4 Hardware-Ereignisüberwachung (detection_hardwareevents)

In diesem Verfahren werden knapp 20 relevante Hardware-Ereignisse kontinuierlich bewertet und mittels eines LSTM-Modells auf anomales Verhalten geprüft.

Das Modell analysiert die Daten in einem gleitenden Zeitfenster. Ergibt die Modellvorhersage eine Angriffswahrscheinlichkeit von über 50 %, wird das Ereignis als Anomalie klassifiziert. Die Ergebnisse, einschließlich der Einzelwahrscheinlichkeiten für *benign* und *attack*, werden über eine dedizierte Schnittstelle an Prometheus exportiert und zur Verfügung gestellt.

4 Konzeption des Threat Reporters

4.1 Zentralisiertes Datenmanagement und Merkmalsextraktion für den Threat Report

Um die Interoperabilität und Standardisierung des Threat Reports zu gewährleisten, wurde entschieden, die erforderlichen Daten zentral aus der Lokalen Prometheus-Datenbank zu extrahieren, anstatt diese einzeln von den jeweiligen Detection-Einheiten abzurufen. Hierfür wurden spezifische Prometheus-Exporter für die Detektionseinheiten entworfen und integriert. Diese stellen die Daten kontinuierlich über die lokalen Ports 2112, 2113, 2115 und 2116 bereit.

Die über die Exporter gelieferten Datensätze umfassen sowohl die Klassifizierung der Anomalien (Attack oder Benign) als auch sämtliche Features, die für die jeweilige Detektion herangezogen wurden. Dieser umfassende Ansatz ist notwendig, da eine zukünftige Optimierung der Gesamtdetektion mittels eines Machine-Learning-Modells vorgesehen ist. Um ein qualitativ hochwertiges Training des Modells zu ermöglichen, kann eine möglichst breite Basis an Inputdaten den Lernprozess signifikant verbessern [3].

Die Grundlage hierfür bildete eine detaillierte Analyse der Detektionseinheiten sowie ein tiefgreifendes Verständnis der bereitgestellten Datenstrukturen, wie bereits in Kapitel 3.3 (Detection Einheiten der Ladeinfrastruktur) dargelegt.

Der Threat Report importiert diese Daten über Prometheus und überführt sie in ein CSV-Format. Das CSV-Format wurde aufgrund seiner weitreichenden Kompatibilität und einfachen Handhabung und Menschenlesbarkeit gewählt. Zusätzlich überschreiten die generierten Thread Reports eine Dateigröße von maximal 30 kB nicht und somit bieten Zeilenbasierte formate wie CSV oder JSON gegenüber spaltenorientierten Formaten wie *Parquet* keine Nachteile. Im Gegenteil Parquet-spezifischer Metadaten-Overhead kann bei derart kleinen Dateien den Großteil der Dateigröße ausmachen, während die Vorteile (Kompression, spaltenbasierter Zugriff) erst bei Dateigrößen im Bereich mehrerer hundert Megabyte relevant werden [4] [2].

4.2 Zeitliche Diskretisierung und Intervalldefinition

Um die Systemressourcen des *Raspberry Pi Zero 2 W* zu schonen und gleichzeitig das Datenaufkommen zu begrenzen, ohne die Detektionssicherheit zu beeinträchtigen, wurde ein Zeitfenster von 60 Sekunden gewählt, das in 30-Sekunden-Intervallen unterteilt ist. Dieser Ansatz ermöglicht es, umfassende Informationen der Detektionseinheiten zu erhalten. Die Parameter für diese Zeitintervalle wurden bewusst variabel gestaltet. Dies schafft die Grundlage für zukünftige Untersuchungen, wie sie in Kapitel 9 (Diskussion und Forschungslücke) beschrieben sind, um den Einfluss unterschiedlicher Zeitfenster auf die Qualität der Reports empirisch zu evaluieren.
Die aktuell implementierte Sequenz umfasst drei Datenpunkte: den aktuellen Zeitpunkt (0 Sekunden) sowie die Zustände der Detektionseinheiten vor 30 und 60 Sekunden. Durch diese Struktur wird eine Timeline generiert, die den zeitlichen Verlauf der Detektion über drei Messpunkte abbildet, welches eine verbesserte Darstellung des zeitlichen Verlaufs ermöglicht als isolierte Zeitpunkte zu betrachten [5] [3].

4.3 Statistische Merkmalsaggregation mittels Sliding-Window-Analyse

Neben den instantanen Merkmalswerten (Features) und Anomalie-Indikatoren der Detektionseinheit werden zusätzlich historische Daten mittels *Sliding Windows* importiert. Pro Threat Report werden drei dieser Fenster definiert, welche jeweils den aktuellen Datenpunkt in Relation zu zwei vorangegangenen Zeitpunkten setzen:

- **Window 1:** 0 Sekunden im Vergleich mit -30 s und -60 s
- **Window 2:** -30 Sekunden im Vergleich mit -60 s und -90 s
- **Window 3:** -60 Sekunden im Vergleich mit -120 s und -150 s

Innerhalb dieser Zeitfenster werden für die statistische Auswertung anhand klassischer Kennzahlen berechnet [3] [1]:

attack scenario

- **Mittelwert (Average):** Der Durchschnittswert innerhalb des Fensters.
- **Extremwerte:** Minimum und Maximum im jeweiligen Intervall.
- **Median:** Der Zentralwert der Datenpunkte.
- **Zuwachs (Increase):** Die relative oder absolute Veränderung innerhalb des Fensters.
- **Summe:** Die kumulierten Werte des jeweiligen Zeitraums.

Durch diese Methodik entsteht ein umfassender Report, der nicht nur eine Momentaufnahme der Detektionseinheiten liefert, sondern auch die zeitliche Entwicklung über die letzten 60 Sekunden bzw. durch die Sliding Windows bis zu 150 Sekunden in die Vergangenheit abbildet. Diese Datenbasis dient als Input für eine spätere Aufbereitung und trainings eines Machine-Learning-Modells, welcher in Kapitel 9 (Diskussion und Forschungslücke) beschrieben sind.
Der für diesen Abschnitt implementierte Quellcode, welcher die Logik der Datenabfrage sowie die Generierung des Reports umfasst, ist im Anhang unter Listing 11.1 (`reporter.py`) und Listing **??** (`queries.py`) dokumentiert.

5 Theoretische Grundlagen der Informationsfusion

5.1 Vergleich von Aggregationsmethoden

5.1.1 Dempster-Shafer-Evidenztheorie

5.1.2 Fuzzy-basierte Fusion

5.1.3 Bayes-basierte Konsensmodelle

5.1.4 Gewichtetes Voting

5.2 State-of-the-Art: Deep Sets und Attention Mechanismen

6 Methodik: Gewichtete Aggregation mittels Utility Theory

6.1 Anforderungen an die Gewichtungsfunktion

6.2 Mathematische Modellierung

6.2.1 Die Gewichtungsformel: $w(n) = BASE_WEIGHT + (MAX_WEIGHT - BASE_WEIGHT) \times (1 - \exp(-K \cdot n))$

NIcht in die überschrift

6.2.2 Parameteranalyse: Monotonie und Diminishing Returns

6.2.3 Abgrenzung zur originären Utility Theory

6.3 Implementierung des Attention-basierten Selektionsmodells

7 Erweiterung der Systemarchitektur und Implementierung

7.1 Generierte Threat Reports: Format und Struktur

7.2 Integration von Threat Reporter und Aggregationslogik

7.3 Architektur der Fusionskomponente im P2P-Netzwerk

8 Evaluation

8.1 Qualität der aggregierten Threat Reports

8.2 Nützlichkeit für die interne Nutzung der Knoten

9 Diskussion und Forschungslücken

9.1 Auswirkungen unterschiedlicher Gewichtungsstrategien auf die Report-Qualität

9.2 Potenzial der LLM-gestützten Aufbereitung für externe CTI-Nutzung

9.3 Untersuchung des Einflusses von Report-Aggregation auf Detectionsergebnisse (Echo-Chamber-Effekt?)

10 Zusammenfassung und Fazit

11 Quellcode

Listing 11.1: reporter.py – Logik der Report-Generierung

```
import time
import requests
import csv
import statistics
from datetime import datetime, timedelta
from queries import get_queries  # liefert NUR Basis-PromQL-Expressions
    zurück

# === KONFIGURATION ===
PROMETHEUS_URL = "http://192.168.0.25"  # Prometheus-Endpunkt
IPFS_API_URL = "http://localhost:3001"  # IPFS API-Endpunkt
OUTPUT_CSV = "threat_report.csv"

# Zeitraum
TIMESPAN = 1  # Minuten
END_TIME = datetime.utcnow()
START_TIME = END_TIME - timedelta(minutes=TIMESPAN)
STEP = "30s"

# Zusätzliche Zeitversätze (in Sekunden)
OFFSETS = [0, -30, -60]


def query_prometheus_range(query, start, end, step):
    """Fragt Prometheus nach einem Zeitbereich (range query)."""
    url = f"{PROMETHEUS_URL}/api/v1/query_range"
    params = {
        "query": query,
        "start": start.isoformat() + "Z",
        "end": end.isoformat() + "Z",
        "step": step,
    }
    response = requests.get(url, params=params)
    response.raise_for_status()
    result = response.json()

    if result["status"] != "success":
        raise RuntimeError(f"Fehler␣bei␣Abfrage:␣{result}")

    return result["data"]["result"]


def compute_basic_stats(values):
    """Berechne avg, min, max, median aus einer Liste von Floats."""
    if not values:
        return None
    try:
        avg = sum(values) / len(values)
        mn = min(values)
```

```
        mx = max(values)
        med = statistics.median(values)
    except Exception:
        return None

    return {"avg": avg, "min": mn, "max": mx, "median": med}


def compute_increase(values):
    """Berechnet eine einfache 'increase' Approximation aus einer Liste
        von Floats."""
    if len(values) < 2:
        return None
    inc = 0.0
    prev = values[0]
    for v in values[1:]:
        delta = v - prev
        if delta > 0:
            inc += delta
        prev = v
    return inc


def main():
    QUERIES = get_queries(START_TIME, END_TIME)

    with open(OUTPUT_CSV, mode="w", newline="", encoding="utf-8") as file
        :
        writer = csv.writer(file)
        # Kopfzeile
        writer.writerow([
            "metric_name",
            "instance",
            "offset",
            "start_time",
            "end_time",
            "num_samples",
            "latest_value",
            "avg",
            "min",
            "max",
            "median",
            "increase",
            "sum"
        ])

        for metric_name, promql in QUERIES.items():
            print(f"Metrik: {metric_name} | Query: {promql}")
            for offset in OFFSETS:
                offset_end = END_TIME + timedelta(seconds=offset)
                offset_start = offset_end - timedelta(minutes=TIMESPAN)

                series_list = query_prometheus_range(promql, offset_start
```

```
                    , offset_end, STEP)

                for series in series_list:
                    metric_labels = series.get("metric", {})
                    instance = metric_labels.get("instance", "unknown")

                    raw_values = []
                    for ts, value_str in series.get("values", []):
                        try:
                            v = float(value_str)
                        except ValueError:
                            continue
                        raw_values.append(v)

                    if not raw_values:
                        continue

                    stats = compute_basic_stats(raw_values)
                    inc = compute_increase(raw_values)
                    total_sum = sum(raw_values)
                    latest_value = raw_values[-1] if raw_values else ""

                    if stats is None:
                        continue

                    writer.writerow([
                        metric_name,
                        instance,
                        f"{offset}s",
                        offset_start.isoformat() + "Z",
                        offset_end.isoformat() + "Z",
                        len(raw_values),
                        f"{latest_value:.6f}",
                        f"{stats['avg']:.6f}",
                        f"{stats['min']:.6f}",
                        f"{stats['max']:.6f}",
                        f"{stats['median']:.6f}",
                        f"{inc:.6f}" if inc is not None else "",
                        f"{total_sum:.6f}",
                    ])

    print(f"CSV-Datei␣erstellt:␣{OUTPUT_CSV}")


if __name__ == "__main__":
    main()
    #time.sleep(TIMESPAN)  # Der Report wird nur einmal pro Minute
        aktualisiert
    #print("Warte auf nächste Aktualisierung...")
    #
    # Hier testweise eine testdatei veröffentlichen
    cid = publish_csv("./testfile.csv", IPFS_API_URL)
```

Bibliography

[1] B. Boswell, S. Barrett, S. Rajaganapathy, G. Dorai, and M. Qiu. FLARE: Feature-based lightweight aggregation for robust evaluation of IoT intrusion detection. *arXiv preprint arXiv:2504.15375*, 2025. Schlägt FLARE vor, eine leichtgewichtige Feature-Aggregationstechnik für IoT-IDS, die mittels zeitbasierter Sliding Windows statistische Kennzahlen wie Flussrate, Paketgrößen, Inter-Arrival-Zeiten und Entropie aggregiert, um kurzlebige Angriffsmuster wie DoS und DDoS zuverlässig zu erkennen.

[2] CelerData. What is parquet file format?, 2024. Erklärt die Architektur des Parquet-Formats, insbesondere die Vorteile der spaltenorientierten Speicherung und Kompression.

[3] M. Heigl, E. Weigelt, A. Urmann, D. Fiala, and M. Schramm. Exploiting the outcome of outlier detection for novel attack pattern recognition on streaming data. *Electronics*, 10(17):2160, 2021. Präsentiert das SOAAPR-Framework, das Ausreißererkennungsergebnisse auf Streaming-Daten auswertet und daraus neuartige Angriffsmuster extrahiert.

[4] R. Taylor-Davies. How good is parquet for wide tables (machine learning workloads) really?, 2023. Analysiert die Performance von Apache Parquet bei Tabellen mit hoher Spaltenanzahl und bewertet den Metadaten-Overhead kritisch.

[5] K. Zhang, F. Zhao, S. Luo, Y. Xin, and H. Zhu. An intrusion action-based IDS alert correlation analysis and prediction framework, 2019. Schlägt das IACF-Framework vor, das Alarme auf Basis intrinsischer Korrelationen zu Intrusion Actions gruppiert und mittels zeitbasierter Sequenzanalyse (TSS) zu Sessions zusammenführt, um mehrstufige Angriffe zu rekonstruieren und vorherzusagen.

Generative KI-Werkzeuge, die in der Arbeit verwendet wurden.

Kapitel	KI-Tool	Version	Anwendung	Erklärung/Kommentar
1.0–4.2	Perplexity AI	2024	Grammatik- und Rechtschreibkorrektion	Der generierte Output wurde ausschließlich zur Überprüfung und Optimierung der Grammatik, Rechtschreibung und Lesbarkeit verwendet. Sämtliche fachlichen Inhalte, Analysen, Interpretationen und wissenschaftliche Schlussfolgerungen stammen vollständig von mir.
1.0–4.2	Perplexity AI	2024	Verbesserung der Lesbarkeit	Umformulierung von Sätzen zur besseren Verständlichkeit und Fluss des Textes, ohne inhaltliche Veränderungen.

Eidesstattliche Erklärung

Ich versichere hiermit wahrheitsgemäß, die Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt, die wörtlich oder inhaltlich übernommenen Stellen als solche kenntlich gemacht und die Satzung der Ostbayerischen Technischen Hochschule Regensburg zur Sicherung guter wissenschaftlicher Praxis in der jeweils gültigen Fassung beachtet zu haben. Die Arbeit ist weder von mir noch von einer anderen Person an der Ostbayerischen Technischen Hochschule Regensburg oder an einer anderen Hochschule zur Erlangung eines akademischen Grades bereits eingereicht worden.

Regensburg, den 01.04.2026 _____

Yannic Hanel

Anomaly Detection Einheit | Anomalydetectioneinheit | usw.. Detection oder Detektion

immer gleich schreiben

immer gleich schreiben

Todo list

Quelle evtl. mit meiner Bachelorarbeit arbeiten 1

Quelle schwierige erkennung wenn netzwerkweite Veränderung und nicht am Knoten selbst 1

Vielleicht finde ich eine begründung warum ich genaue diese events in den hardware report gepackt habe? 2

TODO: Sollte hier wissenschaftlich begründet werden, warum die Wahl auf eine Kombination aus Deep Sets und Attention-Mechanismen gefallen ist? 2

TODO: Die Formel der Utility Theory wurde nicht originär für Gewichtungsfunktionen entwickelt. Wie problematisch ist das? Sollte dies in der Diskussion thematisiert oder direkt eine alternative Methode verwendet werden? 2

TODO: Fokus darauf legen wie die zuvor beschriebenen Methoden in die Architektur einbepflegt wurden 2

Welche Forschungslücke? 2

TODO: Mögliche Forschungslücken: - Untersuchung der Auswirkungen von unterschiedlichen Gewichtungsstrategien Aggregation auf die Qualität der aggregierten Reports - Erweiterung um die LLM gestützte Aufbereitung der Reports für die externe Nutzung - Untersuchung ob der vorgestellte ablauf, erstellen der reports, aggregieren und nutzung der nachbar reports wirklich zu verbesserten Detectionsergebnissen führt (Echo-Chamber-Effekt?). 2

NIcht in die überschrift 14

immer gleich schreiben . . . . . .

immer gleich schreiben . . . . . .